React for Beginners

Components, props, state, and effects

A concise, original learning guide with practical examples and a mini project.

Designed for beginners • 3 learning pages + cover

1. Thinking in Components

React is a JavaScript library for building user interfaces from reusable components. A component describes what should appear for a given set of data.

A simple component

```
function Welcome() {\n  return <h1>Welcome to the library</h1>;\n}\n\nexport default Welcome;
```

JSX looks like HTML but is JavaScript syntax. Return one parent element or use a fragment (<>...</>). Component names begin with a capital letter.

Props

```
function BookCard({ title, author }) {\n  return (\n    <article>\n      <h2>{title}</h2>\n      <p>by
{author}</p>\n    </article>\n  );\n}
```

Props pass information from a parent to a child. Treat props as read-only; a component should not mutate the values it receives.

Render a list

```
const books = [\n  { id: 1, title: "Python", author: "A. Shah" },\n  { id: 2, title: "React", author: "
R. Patil" },\n];\n\n{books.map(book => (\n  <BookCard key={book.id} {...book} />\n))}
```

2. State and Events

State with useState

```
import { useState } from "react";\n\nfunction Counter() {\n  const [count, setCount] = useState(0);\n
return (\n    <button onClick={() => setCount(c => c + 1)}>\n      Clicked {count} times\n    </button>
\n  );\n}
```

State stores values that change over time. Updating state asks React to render again. Do not directly modify state arrays or objects; create updated copies.

Controlled inputs

```
const [query, setQuery] = useState("");\n<input value={query}\n  onChange={event => setQuery(event.targ
et.value)}\n  placeholder="Search books" />
```

A controlled input gets its displayed value from React state. This makes search, validation, and form submission easier to manage.

Conditional UI

```
{books.length === 0 ? (\n  <p>No books found.</p>\n) : (\n  <BookGrid books={books} />\n)}
```

Use conditional rendering to show loading messages, empty states, errors, and content based on current data.

3. Effects and Component Design

useEffect

```
import { useEffect, useState } from "react";\n\nuseEffect(() => {\n  document.title = "E-book Library";
\n}, []);
```

useEffect synchronizes a component with an external system. The dependency array controls when the effect re-runs. Avoid effects for values that can be calculated during rendering.

Practical component structure

Separate the page into focused components such as SearchBar, BookCard, BookGrid, and BookReader. Keep shared data in the closest common parent that needs to coordinate it.

Accessibility and testing

Use semantic HTML, meaningful button labels, alt text for cover images, and visible keyboard focus. Test search behavior, empty results, loading states, and opening a selected book.

Mini project

Create a book catalogue with title, author, category, availability, and a Read Book button. Add a search field and ensure the list updates as the user types.

Start with simple components. Add state and effects only when the feature needs them.